LAPORAN TUGAS 3: API DAN POSTMAN

Book dan Author - Sistem Informasi Penjualan Buku (Book Sales)

Nama: Muhamad Aditia

NIM: 0110224213

Kelas: TI02

1. Tujuan

Mengubah aplikasi Book Sales pada tugas sebelumnya menjadi API. Data Book dan Author ditampilkan dalam format JSON melalui route di routes/api.php, kemudian diuji menggunakan Postman. File view yang sudah tidak diperlukan dihapus.

2. Pemenuhan Instruksi Tugas

No	Instruksi	Pelaksanaan
1	Menggunakan API	Menjalankan php artisan install:api, sehingga Laravel memiliki routes/api.php. Semua route API otomatis berawalan /api.
2	Routing dipindahkan ke routes/api.php	Route Book dan Author dibuat di routes/api.php. File routes/web.php hanya berisi route "/" bawaan.
3	Controller output JSON	BookController dan AuthorController memakai response()->json() dengan format success, message, dan data.
4	Testing dengan Postman	Request GET ke /api/books dan /api/authors menghasilkan status 200 OK (lihat bagian 5).
5	Hapus file view	Folder resources/views/books, authors, dan layouts dihapus. Hanya welcome.blade.php bawaan yang dipertahankan.
6	Push ke GitHub	Proyek diunggah ke repository GitHub pada folder book-sales-3 (lihat bagian 6).

3. Routing (routes/api.php)

Setiap route memanggil method pada Controller. Method index menampilkan seluruh data, sedangkan show menampilkan satu data berdasarkan id.

```
<?php

use App\Http\Controllers\AuthorController;
use App\Http\Controllers\BookController;
use Illuminate\Support\Facades\Route;

// Semua route di file ini otomatis berawalan /api

Route::get('/books', [BookController::class, 'index']);
Route::get('/books/{id}', [BookController::class, 'show']);

Route::get('/authors', [AuthorController::class, 'index']);
Route::get('/authors/{id}', [AuthorController::class, 'show']);
```

4. Controller

BookController mengambil data buku beserta authornya, lalu mengembalikannya sebagai JSON. Jika id tidak ditemukan, method show mengembalikan JSON dengan status 404.

```
<?php

namespace App\Http\Controllers;

use App\Models\Book;
use Illuminate\Http\JsonResponse;

class BookController extends Controller
{
    // GET /api/books
    public function index(): JsonResponse
    {
        $books = Book::with('author')->get();

        return response()->json([
            'success' => true,
            'message' => 'Daftar data buku',
            'data'    => $books,
        ]);
    }

    // GET /api/books/{id}
    public function show(int $id): JsonResponse
    {
        $book = Book::with('author')->find($id);

        if (!$book) {
            return response()->json([
                'success' => false,
                'message' => 'Buku tidak ditemukan',
                'data'    => null,
            ], 404);
        }

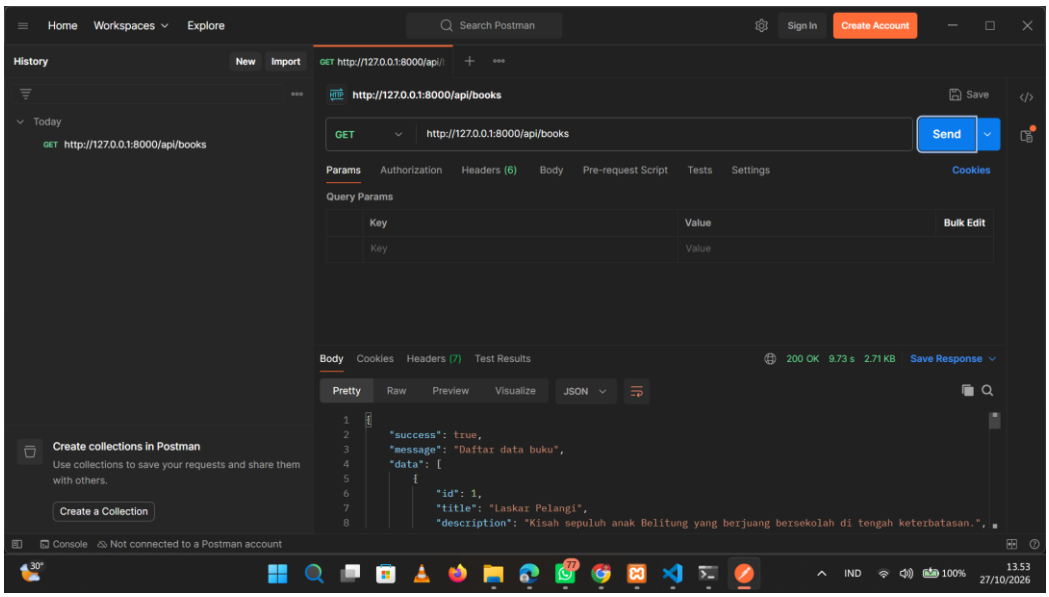
        return response()->json([
            'success' => true,
            'message' => 'Detail data buku',
            'data'    => $book,
        ]);
    }
}
```

AuthorController memiliki struktur yang sama. Method index memakai Author::withCount("books") untuk menampilkan jumlah buku, dan method show memakai Author::with("books") untuk menampilkan daftar buku milik author.

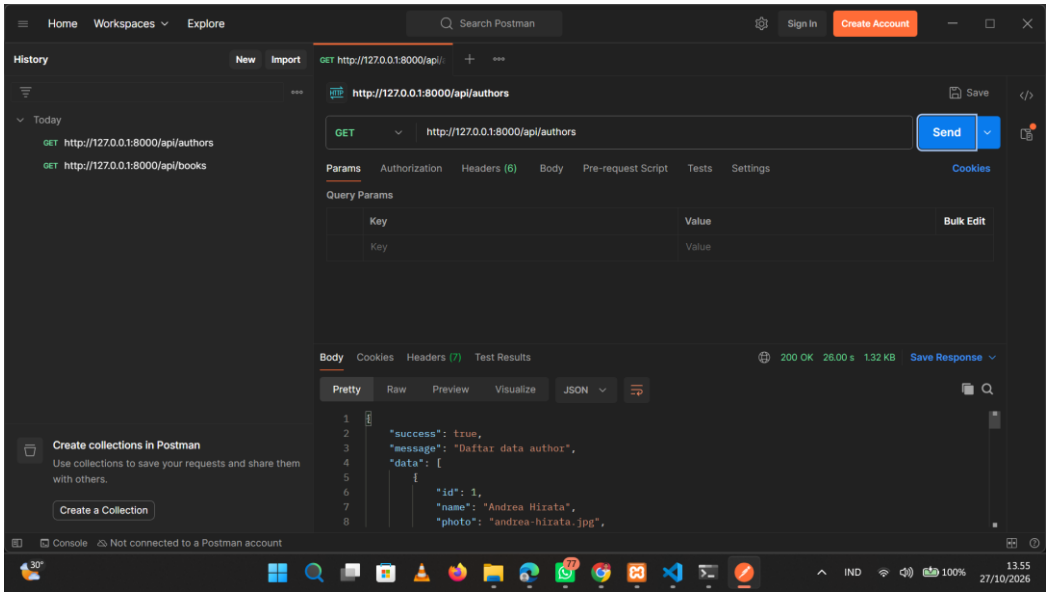
5. Pengujian dengan Postman

Server dijalankan dengan php artisan serve, lalu request GET dikirim melalui Postman.

Method	URL	Status	Hasil
GET	http://127.0.0.1:8000/api/books	200 OK	JSON daftar buku
GET	http://127.0.0.1:8000/api/authors	200 OK	JSON daftar author



Gambar 1. Hasil GET /api/books di Postman



Gambar 2. Hasil GET /api/authors di Postman

6. GitHub

Link repository: https://github.com/muhammadaditia433/Framework-Laravel/tree/main/book-sales-3

File yang dikumpulkan: BookController.php dan AuthorController.php pada folder app/Http/Controllers.

7. Kesimpulan

Dengan API, data aplikasi dikirim dalam format JSON sehingga tidak memerlukan view dan dapat dipakai oleh aplikasi lain, misalnya frontend React pada tahap berikutnya. Postman memudahkan pengujian setiap endpoint tanpa harus membuat tampilan terlebih dahulu.